

TinySSH for Docker Development Environments

Rohit Goswami · 2021-12-26 · tools · workflow

Rootless SSH servers with TinySSH for local GitPod containers

Background

`act` works well enough for most Github Actions workflows, until something fails in the pipeline, or interactive debugging is required. Naturally this is beyond the purview of `act` itself, but it is easy enough to set up shop with a project's GitPod `docker` image being used. For the purposes of the post we'll consider `numpy`.

```
docker run -v $(pwd):/home/gitpod/f2py_skel --name f2py_npdev -it numpy/numpy-dev:latest
```

`docker attach` can be used in tandem with `docker stop` and `docker start`, but these have the disadvantage of only providing one synchronous view of the machine.

An **SSH server** seems like the most natural solution. Rather than cover OpenSSH and its superuser based configuration, we will instead consider the [TinySSH project](#), which works well without superuser access.

TinySSH

We also need to be able to listen for incoming TCP connections and run `tinysshd` in response to a connection. This is a picture perfect scenario for `tcpserver` which is part of the `uscpi-tcp` set of tools.

```
sudo apt install tinysshd uscpi-tcp # for tcpserver
# On focal it is:
# sudo apt install tinysshd ucspi-tcp-ipv6
```

Although this assumes `sudo` access, compiling both `uscpi-tcp` and `tinysshd` from source is a remarkably simple process.

As `tinyssh` [does not implement RSA](#) and older cryptography algorithms, it is easiest to make a new key pair on the host machine.

```
# On the host
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_tiny
ssh-add ~/.ssh/id_ed25519_tiny
# Copy the public key
cat ~/.ssh/id_ed25519_tiny.pub | wl-copy # wayland
# or
cat ~/.ssh/id_ed25519_tiny.pub | xclip # X11
```

`tinyssh` supports only key based authentication, we need to set up the `authorized_keys` correctly.

```
# On the client
mkdir $HOME/.ssh
touch $HOME/.ssh/authorized_keys
chmod 700 .ssh/
echo $COPIED_KEY >> $HOME/.ssh/authorized_keys
chmod 600 .ssh/authorized_keys
```

Finally we can setup server keys and run the server itself, on a port which does not require superuser privileges.

```
tinysshd-makekey $HOME/tinysshkeydir
tcpserver -HRDl0 0.0.0.0 2200 /usr/sbin/tinysshd -v $HOME/tinysshkeydir
```

From the host, we need to know the IP to connect to.

```
docker inspect -f "{{ .NetworkSettings.IPAddress }}" $IMG_NAME
ssh $USER@$IP -p 2200
```

File Permissions

A common issue with `docker` images is that the user and group permissions on the shared folders do not match the permissions within the container. The most obvious fix is to force the `id` by passing `--user $(id -u):$(id -g)` when creating the container. However, a better approach is to change the group and user `id` from within the container itself.

```
id # On the host, note the uid and gid
docker start f2py_npdev
docker attach f2py_npdev
# In docker container
sudo groupmod -g $HOSTGID $USER
sudo usermod -u $HOSTUID $USER
exit # re-enter
```

When re-entering the container, the correct permissions will be set for the shared folder, but any existing files will need to be fixed up. We can also get the `uid` and `gid` of the shared folder directly with `stat`.

```
# in the container
export HOSTGID=$(stat -c "%g" $SHARED_VOLUME)
export HOSTUID=$(stat -c "%u" $SHARED_VOLUME)
export DOCKERUID=$(stat -c "%u" $HOME)
export DOCKERGID=$(stat -c "%g" $HOME)
```

For the fix-up, we can simply use the populated variables.

```
sudo find / -xdev -user "$DOCKERUID" -exec chown -h "$HOSTUID" {} \;
sudo find / -xdev -group "$DOCKERGID" -exec chgrp -h "$HOSTGID" {} \;
```

Conclusions

Like many modern developers, I spend far too long messing around with continuous integration systems and jumping around different development environments. `numpy` and other projects have opted into Gitpod as a lower barrier of entry method to aid contributors. Often the `docker` images backing these do not have `systemd` or service management set up, and so local environments based off of such images can be a little off putting.

Nothing in this post constitutes any kind of best practice. In an ideal world one would have either physical machines to access or well configured virtual machines. Nevertheless, for edge cases, this works well enough in practice .